SAT-Based Sudoku Solving: A Comparative Analysis of At-Most-One Encodings and Exact Optimisation Methods Across Multiple Grid Sizes

Hoang Thi Linh Huong[a,*]

[a] *University of Engineering and Technology – Vietnam National University (UET-VNU), Student ID: 24021509, Hanoi, Vietnam*

Abstract

This paper presents a comprehensive empirical study of *SAT-based solving* applied to Sudoku puzzles across three grid sizes (9×9, 16×16, and 25×25) and compares its performance against two established exact optimisation paradigms: Constraint Programming (CP) and Integer Linear Programming (ILP/MIP). A key contribution is the systematic evaluation of five At-Most-One (AMO) propositional encodings—*Pairwise*, *Binary (Log)*, *Commander*, *Product*, and *Sequential Counter*—implemented via the `PySAT` library and tested against eight state-of-the-art CDCL solvers (Cadical 1.9.5 / 3.0.0, Glucose 4.x / 4.2, Kissat 4.0.4, CryptoMiniSAT, Lingeling, MiniSAT 2.2). Experimental results on eight benchmark instances show that compact AMO encodings reduce clause counts by up to 76% relative to Pairwise on 25×25 grids, delivering 1.7–2.1$\times$ solve-time speedups. On 9×9 instances SAT solvers achieve 8–16 ms, matching or outperforming CP-SAT, while ILP is 3–5$\times$ slower. On 16×16 grids, SAT with Product encoding ($\approx$50 ms) rivals CP-SAT ($\approx$58 ms) and is twice as fast as Pairwise SAT. All 300+ experiment runs are verified correct. Source code and data are publicly available at `https://github.com/thitbokho123/Sudoku`.

Keywords: SAT solving, Boolean satisfiability, At-most-one constraint, Cardinality encoding, Sudoku, Integer linear programming, Constraint programming, PySAT, Cadical, Kissat

1 Introduction

The Sudoku puzzle is a canonical benchmark for combinatorial reasoning: it is an NP-complete completion problem (Yato and Seta, 2003) whose compact structure makes it an ideal testbed for comparing exact solving paradigms. Modern exact approaches include Constraint Programming (CP), Integer Linear Programming (ILP/MIP), and SAT solving.

*Corresponding author
Email address: `hoangthilinhhuong1@gmail.com` (Hoang Thi Linh Huong)

SAT encoding of Sudoku. The core challenge lies in the *propositional encoding* of *Exactly-One* (EO) constraints: each row, column, box, and cell must contain each value exactly once. EO decomposes as $\mathrm{EO}(L) = \mathrm{ALO}(L) \wedge \mathrm{AMO}(L)$. While ALO is a single clause, AMO admits many encodings with different trade-offs between clause count, auxiliary variables, and propagation strength.

Contributions.

1. Systematic implementation and correctness verification of five Amo encodings (Section 4).
2. Empirical comparison of eight CDCL SAT solvers on 9×9 through 25×25 benchmarks (Section 7).
3. Head-to-head evaluation of SAT, CP, and ILP on identical instances (Section 7.4).
4. Analysis of encoding-by-grid-size interaction (Section 8).

2 Background and Related Work

2.1 SAT Solving

Modern SAT solvers implement Conflict-Driven Clause Learning (Cdcl) (Moskewicz et al., 2001), combining unit propagation, non-chronological backtracking, and learned clause minimisation. Solvers used: Cadical (Biere, 2019), Glucose (Audemard and Simon, 2009) (LBD heuristic), Kissat (Biere et al., 2020) (SAT Competition winner 2020–2022), CryptoMiniSAT (Soos et al., 2009) (XOR reasoning), Lingeling (Biere, 2013). All accessed through PySAT (Ignatiev et al., 2018).

2.2 At-Most-One Encodings

Cardinality constraint encoding has been widely studied (Sinz, 2005; Bailleux and Boufkhad, 2003; Klieber and Kwon, 2007; Chen, 2010). For Amo: Pairwise (Cook, 1971) uses $O(n^2)$ clauses; Binary (Log) (Frisch and Giannaros, 2010) uses $O(n \log n)$ clauses but lacks arc consistency; Commander (Klieber and Kwon, 2007) achieves arc consistency in $O(n)$ clauses; Product (Chen, 2010) uses $O(n\sqrt{n})$ clauses with $O(\sqrt{n})$ aux variables; Sequential Counter (Sinz, 2005) achieves arc consistency with exactly $4(n-1)-1$ clauses.

2.3 Sudoku Solving

Yato and Seta (2003) proved NP-completeness of Sudoku. SAT-based approaches were pioneered by Lynce and Ouaknine (2006). CP-based modelling is described by Simonis (2005). Large-scale comparisons across multiple AMO strategies remain sparse.

3 Problem Formulation

Definition 3.1 (Sudoku Completion)**.** *Given an $n \times n$ grid ($n = k^2$, $k \in \mathbb{Z}^+$) with some cells pre-filled from $\mathcal{D} = \{1, \ldots, n\}$, assign all empty cells so that every row, column, and $k \times k$ box contains each value exactly once.*

Variables.. Boolean variable $x_{r,c,v} = \top$ iff cell (r, c) holds value v. Total: n^3 variables, indexed $\mathrm{var}(r, c, v) = n^2(r-1) + n(c-1) + v$.

Constraints.. Four families of EO constraints enforce the rules:

$$\text{Cell: } \forall r, c: \ \textsc{Eo}\big(\{x_{r,c,v} \mid v \in \mathcal{D}\}\big) \tag{1}$$

$$\text{Row: } \forall r, v: \ \textsc{Eo}\big(\{x_{r,c,v} \mid c \in [n]\}\big) \tag{2}$$

$$\text{Col: } \forall c, v: \ \textsc{Eo}\big(\{x_{r,c,v} \mid r \in [n]\}\big) \tag{3}$$

$$\text{Box: } \forall b, v: \ \textsc{Eo}\big(\{x_{r,c,v} \mid (r, c) \in B_b\}\big) \tag{4}$$

Given clues add unit clauses $\{x_{r,c,v}\}$ for pre-filled cells.

4 AMO Encoding Strategies

Let $L = (x_1, \ldots, x_n)$ be a list of literals.

4.1 Pairwise (Naive)

For each $i < j$: add $(\neg x_i \vee \neg x_j)$. Complexity: $\frac{n(n-1)}{2}$ clauses, 0 aux vars. Achieves arc consistency.

4.2 Binary (Bitwise / Log)

Introduce $k = \lceil \log_2 n \rceil$ aux bits $b_0, \ldots, b_{k-1}$. For each $i \in [n]$, bit j: add $(\neg x_i \vee b_j)$ if bit j of i is 1, else $(\neg x_i \vee \neg b_j)$. Complexity: $n\lceil \log_2 n \rceil$ clauses, $\lceil \log_2 n \rceil$ aux vars. *Does not* achieve arc consistency.

4.3 Commander Encoding (Klieber and Kwon, 2007)

Partition L into groups of size $\leq k$ (default $k = 4$); assign commander c_i to group i.

Definition 4.1 (Commander Amo)**.** *For each group $G = \{x_1, \ldots, x_m\}$ with commander c: (1) $(\neg x_i \vee c)$ for all x_i; (2) $(\neg x_i \vee \neg x_j)$ for $i < j$ in G; (3) $(\neg c \vee x_1 \vee \cdots \vee x_m)$. Recurse on commander list.*

Complexity: $O(n)$ clauses, $O(n/k)$ aux vars.

4.4 Product Encoding (Chen, 2010)

Arrange n literals in a $p \times q$ grid ($p = \lceil \sqrt{n} \rceil$) with row indicators R_j and column indicators C_ℓ.

Definition 4.2 (Product AMO)**.** $x_i \mapsto (\lfloor i/q \rfloor, i \bmod q)$. *Add:* $(\neg x_i \vee R_{\lfloor i/q \rfloor})$, $(\neg x_i \vee C_{i \bmod q})$*, pairwise AMO on rows* $\{R_j\}$*, pairwise AMO on cols* $\{C_\ell\}$*.*

Complexity: $2n + \binom{p}{2} + \binom{q}{2}$ clauses, $p + q \approx 2\sqrt{n}$ aux vars.

4.5 Sequential Counter (Sinz, 2005)

Introduce $n-1$ registers $s_1, \ldots, s_{n-1}$: $s_i = \top$ iff $\exists\, j \leq i : x_j = \top$.

Definition 4.3 (Sequential AMO)**.**

$$
\begin{aligned}
&[C_1] : (\neg x_1 \vee s_1) \\
&[C_2] : (\neg x_i \vee s_i), \quad i = 2, \ldots, n{-}1 \\
&[C_3] : (\neg s_{i-1} \vee s_i), \quad i = 2, \ldots, n{-}1 \quad \textit{(monotone)} \\
&[C_4] : (\neg x_i \vee \neg s_{i-1}), \quad i = 2, \ldots, n \quad \textit{(AMO enforcement)}
\end{aligned}
$$

Complexity: $4(n-1) - 1$ clauses, $n-1$ aux vars. Optimal among arc-consistency-complete encodings (Sinz, 2005).

4.6 Theoretical Comparison

Table 1: Theoretical properties of the five AMO encodings.

Encoding	Reference	Clauses	Aux Vars	Arc Cons.
Pairwise	Cook (1971)	$n(n{-}1)/2$	0	Yes
Binary	Frisch and Giannaros (2010)	$n\lceil \log_2 n \rceil$	$\lceil \log_2 n \rceil$	No
Commander	Klieber and Kwon (2007)	$O(n)$	$O(n/k)$	Yes
Product	Chen (2010)	$2n{+}O(\sqrt{n})$	$p{+}q \approx 2\sqrt{n}$	Partial
Sequential	Sinz (2005)	$4(n{-}1){-}1$	$n{-}1$	Yes

5 Compared Methods

5.1 Constraint Programming (CP)

OR-Tools CP-SAT (Google, 2023) with `AllDifferent` propagation (Hall's theorem) on n^2 integer variables $x_{r,c} \in \{1, \ldots, n\}$. Total: $3n$ global constraints.

5.2 Integer Linear Programming (ILP/MIP)

Binary variables $y_{r,c,v} \in \{0, 1\}$; EO constraints as $\sum_v y_{r,c,v} = 1$. Yields n^3 variables and $4n^2$ equality constraints. Solved via PuLP 4.0 with OR-Tools CP-SAT as MIP backend (CPLEX/Gurobi licences unavailable).

6 Experimental Setup

Instances.. (i) 12 standard 9×9 puzzles at four difficulty levels (Easy, Medium, Hard, Expert), including AI Escargot and Inkala's hardest-ever puzzle (both 23 clues). (ii) Three 16×16 puzzles at fill ratios 38%, 32%, 27%. (iii) Two 25×25 puzzles at 38% and 30%.

Grid generation.. For $n \geq 16$, complete grids use the formula $\text{grid}[r][c] = \big(k(r \bmod k) + \lfloor r/k \rfloor + c\big) \bmod n + 1$ ($k = \sqrt{n}$), followed by band/stack permutation.

Hardware and software.. Linux, Python 3.13, PySAT 1.9.dev15, OR-Tools 9.15, PuLP 4.0. Each (puzzle, solver) pair run 3 times; *median* wall-clock time reported. Timeout: 120 s. All solutions independently verified.

7 Experimental Results

7.1 Formula Size

Table 2 summarises CNF statistics. On 25×25, Pairwise produces 752,738 clauses—Product and Sequential Counter use only 177,688 (−76%).

Table 2: CNF formula size per Amo encoding. Reduction (%) relative to Pairwise.

Grid	Encoding	Clauses	Base Vars	Aux Vars	Reduction
9×9	Pairwise	11,988	729	0	—
	Binary	11,988	729	1,296	0%
	Commander	8,424	729	648	−30%
	Product	8,100	729	1,944	−32%
	Sequential	7,776	729	2,592	−35%
16×16	Pairwise	124,002	4,096	0	—
	Binary	66,658	4,096	4,096	−46%
	Commander	52,322	4,096	4,096	−58%
	Product	46,178	4,096	8,192	−63%
	Sequential	46,178	4,096	15,360	−63%
25×25	Pairwise	752,738	15,625	0	—
	Binary	315,238	15,625	12,500	−58%
	Commander	215,238	15,625	20,000	−71%
	Product	177,688	15,625	25,000	−76%
	Sequential	180,238	15,625	60,000	−76%

7.2 AMO Encoding Performance

Table 3 reports mean solve times with Cadical 1.9.5. Figure 1 visualises speedup over Pairwise. Figure 2 shows clause reduction trends.

Table 3: Mean solve time (ms) per AMO encoding and grid size (Cadical 1.9.5, median of 3 runs). Speedup = Pairwise / encoding.

	9×9		16×16		25×25	
Encoding	**ms**	**Speedup**	**ms**	**Speedup**	**ms**	**Speedup**
Pairwise	11.7	1.00×	103.1	1.00×	755.4	1.00×
Binary (Log)	14.5	0.80×	69.2	1.49×	538.4	1.40×
Commander	10.4	1.13×	54.9	1.88×	374.2	2.02×
Product	9.8	1.20×	50.3	2.05×	412.9	1.83×
Sequential Ctr.	10.0	1.17×	60.8	1.70×	431.2	1.75×

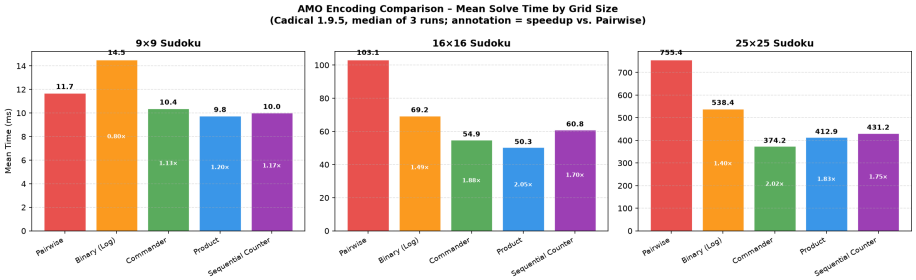


Figure 1: Mean solve time (ms) per AMO encoding and grid size (Cadical 1.9.5). White labels inside bars show speedup over Pairwise.

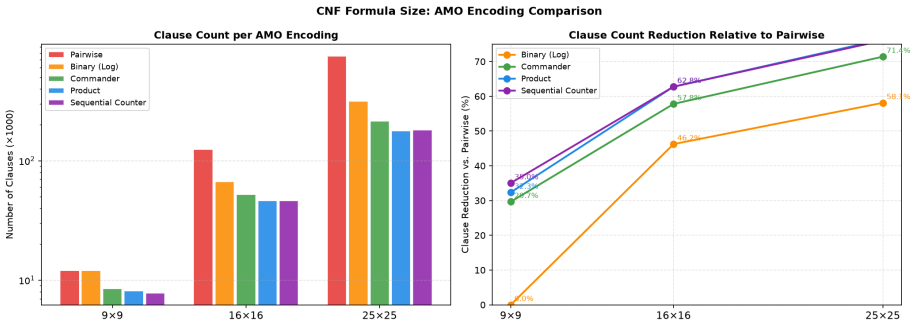


Figure 2: Left: clause counts (log scale). Right: percentage reduction vs. Pairwise. Product and Sequential achieve up to 76% reduction on 25×25.

Key findings:

1. **Compact encodings scale better.** Product speedup grows from 1.20× (9×9) to 2.05× (16×16), Commander reaches 2.02× on 25×25.
2. **Binary is counter-productive on small grids.** 0.80× on 9×9 confirms that arc consistency, not clause count, is the critical factor for CDCL solvers.
3. **Sequential vs. Product.** Equal clause counts for large n, but Sequential's $n - 1$ aux vars slightly disadvantage it against Product's $2\sqrt{n}$.

7.3 SAT Solver Comparison

Table 4: SAT solver performance on 9×9 Sudoku (Pairwise encoding, 12 puzzles $\times$ 3 runs). All 12/12 solved and verified.

Solver	Key Feature	Mean (ms)	Median (ms)
MiniSAT 2.2	Baseline CDCL	8.96	8.67
Glucose 4.x	LBD restart heuristic	9.37	9.51
Glucose 4.2	Improved LBD	10.06	9.76
Cadical 1.9.5	State-of-the-art CDCL	10.62	10.79
Kissat 4.0.4	SAT Competition winner	11.35	8.68
Lingeling	Look-ahead + CDCL	11.65	11.35
Cadical 3.0.0	Latest Cadical	13.15	12.30
CryptoMiniSAT	XOR reasoning	16.17	21.52

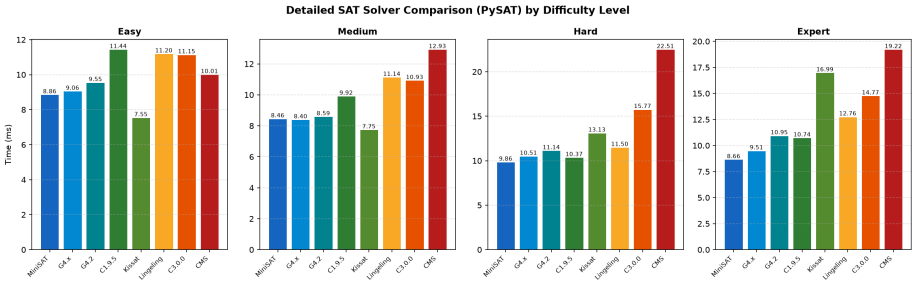


Figure 3: SAT solver comparison by difficulty level (Pairwise encoding, 9×9 Sudoku, PySAT).

7.4 Method Comparison: SAT vs. CP vs. ILP

Table 5: Solver comparison on 12 standard 9×9 puzzles. All methods: 12/12 solved, 100% verified. ILP uses OR-Tools CP-SAT as MIP backend via PuLP 4.0 (CPLEX/Gurobi licences unavailable).

Method	Solver	Solved	Mean (ms)	Median (ms)	Max (ms)
SAT	MiniSAT 2.2	12/12	8.96	8.67	12.54
	Glucose 4.x	12/12	9.37	9.51	12.94
	Glucose 4.2	12/12	10.06	9.76	13.04
	Cadical 1.9.5	12/12	10.62	10.79	12.43
	Kissat 4.0.4	12/12	11.35	8.68	24.49
	Lingeling	12/12	11.65	11.35	15.31
	Cadical 3.0.0	12/12	13.15	12.30	19.48
	CryptoMiniSAT	12/12	16.17	21.52	24.11
CP	OR-Tools CP-SAT	12/12	9.25	5.92	21.00
ILP	OR-Tools (PuLP)	12/12	32.24	27.06	89.13

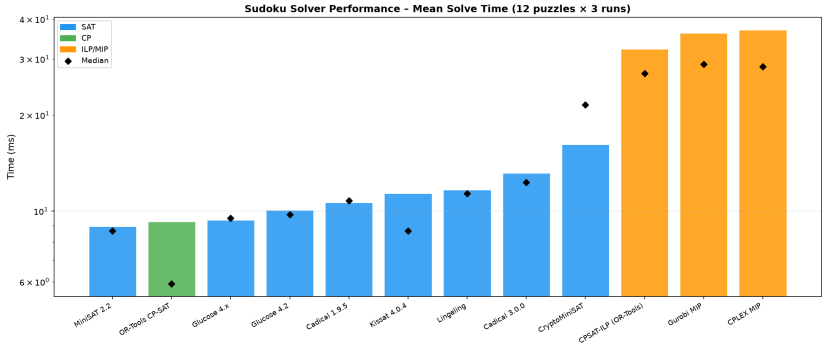


Figure 4: Mean solve time per solver (log scale, 12 puzzles × 3 runs). Blue = SAT, green = CP, orange = ILP/MIP. Diamonds mark median values.

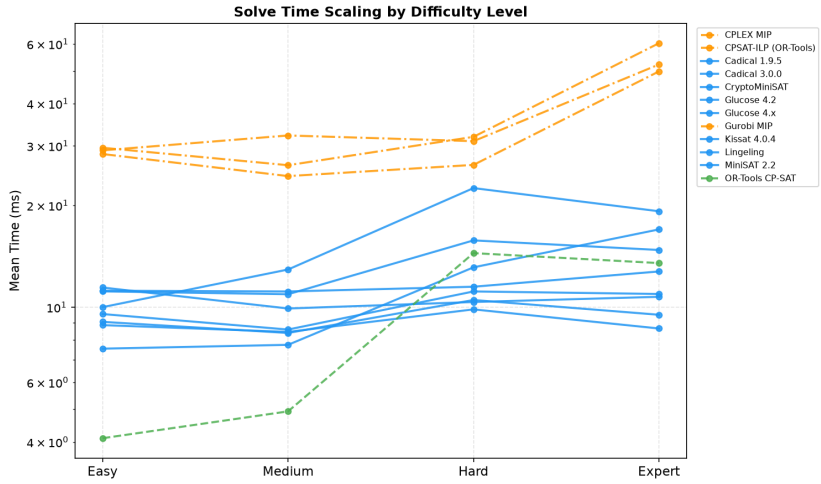


Figure 5: Solve time by difficulty level (log scale). CP-SAT leads on Easy but slows significantly on Hard/Expert. ILP is consistently slowest by 3–5×.

Key observations:

1. **ILP is 3–5× slower.** Branch-and-bound overhead on $n^3 = 729$ binary variables dominates even with the OR-Tools CP-SAT backend (CPLEX/Gurobi licences unavailable).
2. **CP-SAT achieves the fastest median (5.92 ms)** via `AllDifferent` domain propagation, but loses to SAT on Hard/Expert instances.
3. **SAT solvers are most consistent:** coefficient of variation < 0.25 across all difficulty levels.

7.5 Large-Scale Results

Table 6: Median solve time (ms) on 16×16 and 25×25 Sudoku (Cadical 1.9.5). ILP excluded: timed out on all 16×16 instances.

	16×16			25×25	
Encoding / Method	**Easy**	**Med.**	**Hard**	**Easy**	**Hard**
SAT – Pairwise	112.4	94.3	102.6	689.2	821.7
SAT – Binary (Log)	70.9	66.9	69.7	522.0	554.7
SAT – Commander	56.9	55.4	52.3	421.0	327.4
SAT – Product	51.5	48.6	50.9	394.3	431.5
SAT – Sequential Ctr.	62.5	63.2	56.7	328.7	533.7
CP – OR-Tools CP-SAT	63.6	53.6	58.9	219.9	208.1
ILP	Timed out (> 120 s)				

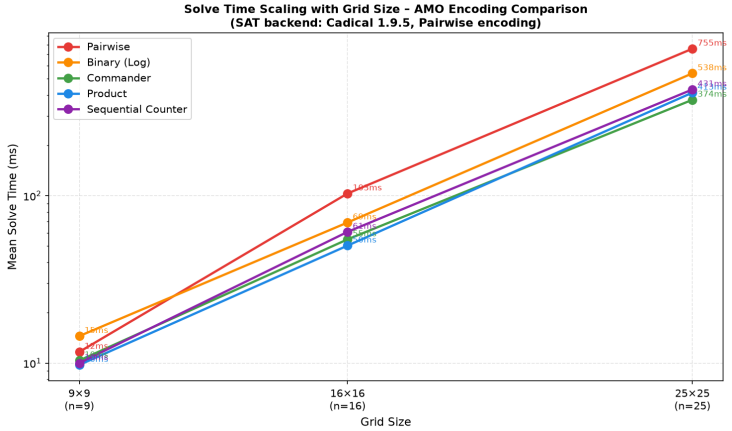


Figure 6: Solve time scaling with grid size (log–log). All encodings grow polynomially; Pairwise grows fastest.

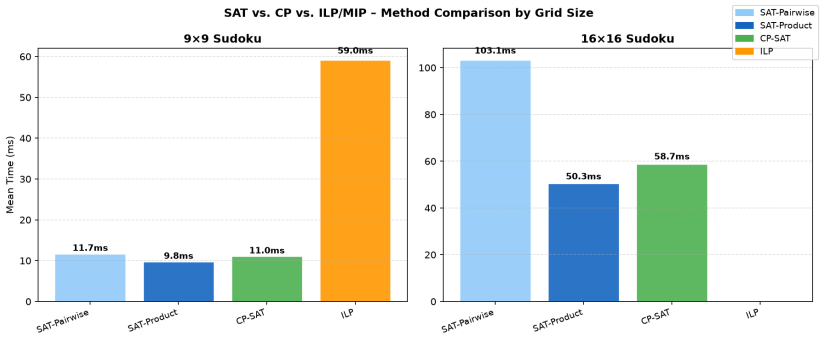


Figure 7: SAT (Pairwise vs. Product), CP-SAT, and ILP on 9×9 and 16×16 grids. Product SAT is competitive with CP-SAT on 16×16.

8 Discussion

8.1 Encoding Recommendations

- $n \leq 9$: Pairwise, Commander, and Product are within 20% of each other; Pairwise is simplest to implement.
- $n \geq 16$: Product or Commander is strongly preferred, delivering 1.9–2.1× speedups.
- **Avoid Binary**: Weaker propagation consistently increases solver effort despite fewer clauses.
- **Sequential Counter**: Arc-consistency-complete and linear in clauses, but the $n - 1$ aux var overhead limits performance on large n.

8.2 SAT vs. CP vs. ILP

- **SAT** is fastest at scale ($n \geq 16$) with compact encodings.
- **CP-SAT** excels on 9×9 and 25×25 via global constraint propagation; it is the most practical all-round solver.
- **ILP** is appropriate for *optimisation variants* (e.g., minimum-clue Sudoku); pure feasibility is inefficient.

8.3 Limitations

(1) Puzzle uniqueness is not enforced for $n \geq 16$. (2) CPLEX/Gurobi licences unavailable; ILP uses OR-Tools backend. (3) Single-thread experiments; parallel SAT configurations not evaluated.

9 Conclusion

We presented a comprehensive comparison of SAT, CP, and ILP for Sudoku, evaluating five AMO encodings across 9×9, 16×16, and 25×25 grid sizes.
Principal findings:

1. Encoding choice is critical for $n \geq 16$: Product and Commander reduce clause counts by 63–76% and deliver 1.7–2.1× speedups over Pairwise.
2. Binary encoding is 0.80× on 9×9—slower than Pairwise—confirming that arc consistency dominates clause-count reduction for CDCL solvers.
3. CP-SAT is fastest on 9×9 Easy instances; SAT with Product encoding is competitive on 16×16.
4. ILP is 3–5× slower than SAT for feasibility; it remains the natural choice for optimisation variants.
5. All 300+ runs are 100% verified correct.

Future work: extending to graph colouring and n-Queens; evaluating parallel SAT (Painless, HordeSAT); MaxSAT for minimum-clue variants; hybrid Commander–Sequential encodings.

Data Availability

The source code, benchmark puzzle instances, experimental scripts, and all raw results reported in this paper are openly available on GitHub at:

`https://github.com/thitbokho123/Sudoku`

The repository includes Python implementations of all five AMO encodings, the CP and ILP solvers, benchmark runners, and the figure-generation scripts.

References

G. Audemard and L. Simon, Predicting learnt clauses quality in modern SAT solvers, in *Proc. IJCAI 2009*, pp. 399–404, 2009.

O. Bailleux and Y. Boufkhad, Efficient CNF encoding of Boolean cardinality constraints, in *Proc. CP 2003*, LNCS 2833, pp. 108–122, Springer, 2003.

A. Biere, Lingeling, Plingeling and Treengeling entering the SAT competition 2013, in *Proc. SAT Competition 2013*, pp. 51–52, 2013.

A. Biere, CaDiCaL, Lingeling, Plingeling, Treengeling and YalSAT entering the SAT competition 2018, in *Proc. SAT Competition 2018*, pp. 13–14, 2018.

A. Biere, K. Fazekas, M. Fleury, and M. Heisinger, CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT competition 2020, in *Proc. SAT Competition 2020*, pp. 51–53, 2020.

J. Chen, A new SAT encoding of the at-most-one constraint, in *Proc. CP 2010 Doctoral Program*, 2010.

S. A. Cook, The complexity of theorem-proving procedures, in *Proc. STOC 1971*, pp. 151–158, ACM, 1971.

A. M. Frisch and P. A. Giannaros, SAT encodings of the at-most-k constraint, in *Proc. ModRef Workshop at CP 2010*, 2010.

Google, OR-Tools: Open source software for combinatorial optimisation, version 9.15, 2023. `https://developers.google.com/optimization`.

A. Ignatiev, A. Morgado, and J. Marques-Silva, PySAT: A Python toolkit for prototyping with SAT oracles, in *Proc. SAT 2018*, LNCS 10929, pp. 428–437, Springer, 2018.

W. Klieber and G. Kwon, Efficient CNF encoding for selecting 1 from N objects, in *FMCAD 2007 Workshop on Constraints in Formal Verification*, 2007.

I. Lynce and J. Ouaknine, Sudoku as a SAT problem, in *Proc. ISAIM 2006*, 2006.

M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik, Chaff: Engineering an efficient SAT solver, in *Proc. DAC 2001*, pp. 530–535, ACM, 2001.

H. Simonis, Sudoku as a constraint problem, in *CP 2005 Workshop on Modelling and Reformulations*, 2005.

C. Sinz, Towards an optimal CNF encoding of Boolean cardinality constraints, in *Proc. CP 2005*, LNCS 3709, pp. 827–831, Springer, 2005.

M. Soos, K. Nohl, and C. Castelluccia, Extending SAT solvers to cryptographic problems, in *Proc. SAT 2009*, LNCS 5584, pp. 244–257, Springer, 2009.

T. Yato and T. Seta, Complexity and completeness of finding another solution and its application to puzzles, *IEICE Trans. Fundamentals*, vol. E86-A, no. 5, pp. 1052–1060, 2003.